MMA 616 · MANAGING INTELLIGENCE · DAY 2

Build: Developing the Agentic Infrastructure

Build the AI capabilities your app needs, assemble them into a working app, and ship it into a week of real use.

Sat June 27, 2026 · Room ESB-236 · Dr. Vern L. Glaser

TODAY

Day 2 at a glance: four blocks

Block 1

Why capabilities, and which you need

Why they beat plain chat; map what your app must do and pick a form for each

Block 2

How to build them

Skills, subagents, connectors, and dynamic workflows

Block 3

Build and test the app

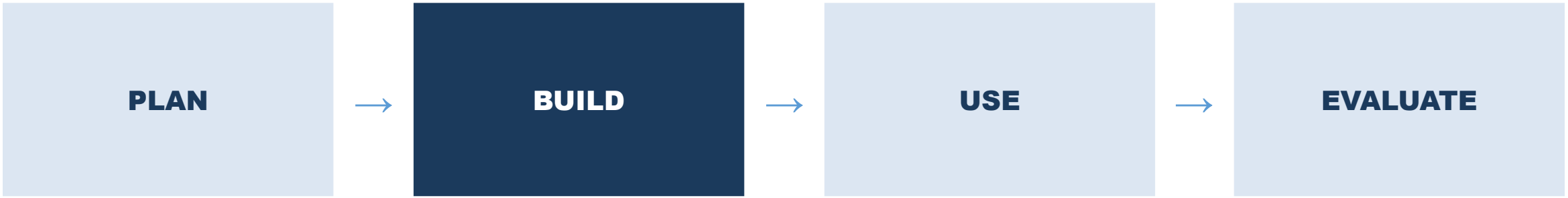
Have Claude build it from your spec, then test it by using it

Block 4

Ship into real use

Put it on GitHub, deploy it, and run it for a week

Where today fits: the Build step



Agentic Project Design runs a four-step loop — Plan, Build, Use, Evaluate — around one written spec. Day 1 was Plan: you decided what to build and wrote the spec (your CLAUDE.md file). Today is Build: you turn that spec into a working, deployed app. The app then goes into a week of real Use, and on Day 3 you Evaluate it against the spec.

WHAT DAY 1 HANDED YOU

You start with a mock-up that looks finished but does nothing

- **Day 1 left you three things.** Curated Knowledge (your data, connected, plus the notes that matter); a written spec called CLAUDE.md that a stranger could build from; and an HTML mock-up of the app.
- **That mock-up is a "walking skeleton."** It shows the real screens and layout, but every number and answer in it is faked. It looks like the product; nothing actually works yet.
- **Build's job is narrow.** Replace the faked content with real, working capabilities, then put the app somewhere people can open it. That is all today is.
- **Today's two questions.** Which capabilities does the app actually need, and how do you build each one? The next slides answer both, in that order.

> **By the end of today the faked mock-up becomes a working app people can use.**

IDENTIFY

CHAPTER 5

Identifying the Capabilities You Need

First understand why capabilities beat plain chat, then map what your app must do and choose a form for each.

Why build capabilities instead of just chatting with the agent

- **Plain chat re-explains and forgets** In an ordinary chat you restate the rules every time you start, and the agent has no way to enforce them. Over a long thread it loses earlier instructions, so the same job gets done a slightly different way each run.
- **Plain chat can only talk** An ordinary chat can reason and write, but it cannot reach your real data or tools. It answers from whatever you pasted in, not from the live file, system, or check the job actually depends on.
- **A capability fixes both gaps** A capability packages the expertise so the agent does the job the same correct way every run, and can reach real systems. A skill packages a procedure or check; a subagent is a scoped worker for one decision; a connector reaches an outside system.
- **The risk is leaving this unused** For a room that can already build, the mistake is almost never over-building. It is staying in plain chat and shipping an app that makes avoidable mistakes, when a small skill or check would have prevented them every time.

> **Capabilities turn one-off chatting into an app that does the job the same right way on every run.**

EXAMPLE

Reading a company's annual report without grabbing the wrong number

An analyst builds a small app: drop in a company's annual report (a 10-K, the long yearly filing public companies file with regulators), ask for a figure such as net revenue, and get the answer. The job is to get the same right number every run. The trouble is that a 10-K is 100-plus pages with dozens of tables full of dollar figures, and the right number hides among convincing wrong twins.

With plain chat you paste the PDF and ask 'what's net revenue?' The model finds a revenue figure and answers confidently — but it may grab one from the wrong place: a marketing highlight, last year's column or a since-corrected (restated) figure, a single business segment's subtotal rather than the company-wide (consolidated) total, an 'adjusted' non-official (non-GAAP) figure instead of the audited one, or a table labeled 'in thousands' read as plain dollars (off by 1,000x). Run it next quarter on another company and the behavior drifts.

Wrap the same job as a skill — a small named package the agent loads and runs every time — that encodes the rule: read only the audited consolidated statements, confirm the fiscal year and the units, and flag any restatement, with a check in code that re-reads the figure from the source before answering. Now the app returns the right number every run, with the standard enforced rather than hoped for. (Figures here are illustrative, not a real company.)

> **A confident number is a claim, not a fact. A skill writes the standard down once and enforces it on every run.**

THE FIVE CAPABILITY FORMS

Five ways to supply a capability — pick the simplest

Native action	Build nothing: the model (Claude in Claude Code) already does it well, so wrapping it only adds a surface that can fail. In Kinquiry, reading the survey file is native action.
Skill	A small named bundle of instructions, optionally with a code check, that the agent loads and runs the same way every time. Choose it when one judgment recurs. Kinquiry's codebook-guard is a skill.
Subagent	A separate helper that runs in its own context and returns only its result. Choose it when one distinct decision deserves its own attention. Kinquiry's hypothesis-parser is a subagent.
Connector	A wired-in link to an outside tool or data source. Thousands exist, so check and configure first; do not hand-build what you could set up. Kinquiry left this form empty on purpose.
Fixed pipeline	Hard-coded steps in a set order, so the same inputs always give the same outputs — used where that protects the result. Kinquiry's analysis runs clean to test to effect size to reliability to robustness.

A menu, not a ranking. For each capability pick the simplest form that does the job — the cheapest capability is the one you do not build.

MOVE 1

Decompose into capabilities

List everything the app must do as plain verbs, before you name a single tool.

MOVE 1 OF 2 · DECOMPOSE

List what the app must do as plain verbs, before naming any tool

- **Separate the need from the how** A capability is something the app must be able to DO — read the data, validate a column, run the test, write the claim — stated as a plain verb. A form (skill, subagent, connector) is HOW you supply it. This move names the verbs only; forms come next.
- **Walk the skeleton; faked spots are verbs** Go screen by screen through your walking skeleton (the HTML mock-up with every piece of content faked — hardcoded numbers, no real data wired in). On Kinquiry's results screen the coefficient was a hardcoded fake; the verb behind it is 'run the analysis,' and the faked caveat text is 'interpret with caveats.'
- **Name the need, not the tool** Write 'we'll need a skill here' and you have fixed HOW before stating WHAT — then the list grows to fit tools you know, not the job. 'We need a statistics connector' is just the verb 'run the analysis' (and no connector turns out to be needed).

> **You have decomposed well when every item names something the app DOES, with no tool, skill, subagent, or connector anywhere on the list.**

KINQUIRY · THE CAPABILITY LIST

Kinquiry's six verbs — named before any tool is chosen

Kinquiry, the instructor's research workbench on the STEP family-business survey (2,683 firms), must do exactly six things. Each is a plain verb; not one names a tool yet.

1. **Read the survey file**
2. **Validate any column against the codebook (the survey's data dictionary of 145 real columns)**
3. **Turn a rough hypothesis into a checked plan — or an honest refusal**
4. **Run the chosen analysis**
5. **Interpret the result with the required caveats**
6. **Render the result as a research presentation**

> **Six verbs, no more. Choosing a form for each comes next, on the capability-to-form map.**

MOVE 2

Choose a form for each

Go down the list and pick the simplest way to build each job.

MOVE 2 OF 2 · CHOOSE A FORM

For each verb, choose the simplest form that does the job

- **Take one verb at a time** Work down your verb list one item at a time. For each, you are choosing a form — the way you supply that capability. There are five forms, ordered from no machinery to the most: native action, skill, subagent, connector, fixed pipeline.
- **Each form answers one question** Ask in order: Can the model already do it well? → native action (build nothing). Does this exact judgment recur and need to hold every time? → skill. Is it one distinct decision worth its own attention? → subagent. Does it need live outside data or a tool? → connector. Must identical inputs always give identical outputs? → fixed pipeline.
- **Pick the cheapest that works** Stop at the first form that does the job; the cheapest capability is the one you don't build. The skill of this move is making each call confidently — and, once, naming a form you were tempted to build and chose not to, with the reason.

> **You have chosen well when every verb sits beside one chosen form, on the simplest form that does its job.**

What we chose for Kinquiry: each verb beside its form

READ SURVEY

→ Native action

The model already reads a local file well, so nothing needs building.

VALIDATE COLUMNS

→ Skill: codebook-guard

A rule that must hold every run, so a skill with a code check enforces it.

PARSE HYPOTHESIS

→ Subagent: hypothesis-parser

One distinct decision — is this answerable, and how? — so its own scoped worker.

RUN ANALYSIS

→ Fixed pipeline

Must be reproducible, so the steps are hard-coded in one fixed order.

INTERPRET

→ Subagent: interpreter

A second distinct decision — what may we honestly claim? — so its own worker.

RENDER PRESENTATION

→ Skill: presentation-render

A repeatable procedure with no judgment in it, so a skill, not a worker.

Three skills, two subagents, one fixed pipeline — and the connector form left empty on purpose, because Kinquiry reads a local file and writes a local file with no live outside system to reach.

DEVELOP

CHAPTER 6

Building the Capabilities

Now build each capability — and learn how skills, subagents, connectors, and workflows actually work.

THE BUILDING BLOCKS

The pieces you will assemble to build capabilities

	Subagents	Skills	Agent teams	Workflows
What it is	A worker Claude spawns	Instructions Claude follows	A lead agent supervising peer sessions	A script the runtim executes
Who decides what runs next	Claude, turn by turn	Claude, following the prompt	The lead agent, turn by turn	The script
Where intermediate results live	Claude's context window	Claude's context window	A shared task list	Script variables
What's repeatable	The worker definition	The instructions	The team definition	The orchestration itself
Scale	A few delegated tasks per turn	Same as subagents	A handful of long-running peers	Dozens to hundred of agents per run
Interruption	Restarts the turn	Restarts the turn	Teammates keep running	Resumable in the same session

Skill: a packaged procedure (codebook-guard). Subagent: a scoped worker with its own context (hypothesis-parser). Workflow: scaffolding for one task (the dashboard).

FORM 1 OF 4

Claude Skills

A reusable package of expertise the agent can find, load, and run on its own.

FORM 1 OF 4 · WHAT A CLAUDE SKILL IS

A Claude Skill is a small folder the agent finds, loads, and runs

- **A folder with one required file** A Claude Skill is a folder containing a file named SKILL.md, plus any optional helper files. SKILL.md opens with a short header written in YAML (a plain key-and-value text format) that has two required fields: a name, and a description that states both what the skill does and when the agent should use it. Below the header are the markdown instructions the agent follows when it runs the skill.
- **It can bundle scripts and reference files** The same folder can hold a small script the agent runs, and reference files it reads only if needed. Kinquiry's codebook-guard skill is exactly this: a SKILL.md, a scripts/validate.py that checks columns, and a references/columns.json listing the survey's 145 real columns.
- **Progressive disclosure keeps it cheap** The agent keeps only each skill's one-line description loaded at all times. It opens the full SKILL.md only when a task matches that description, and reads the bundled files only if the job needs them. So you can install many skills without slowing the agent down.
- **Why it beats a pasted prompt** A pasted prompt is text you retype each time and hope sticks. A skill is discoverable (the agent finds it by its description), reusable across chats and projects, and can carry a check that runs in code. A prompt only advises; codebook-guard's validate.py exits with an error and blocks the run, so the rule is enforced, not suggested.

> **A skill turns expertise that must hold every time into a package the agent loads on its own and a check it cannot skip.**

You do not have to write a skill by hand — Claude can scaffold it

1. **Use the skill-creator skill** Anthropic ships a skill whose job is making other skills. In Claude, ask it to build one and describe in plain words what you want the skill to do and when it should fire. It does not run your skill — it writes the files.

2. **Answer its questions; review what it drafts** It asks clarifying questions, then generates a properly-formatted SKILL.md (the YAML name and description plus the instructions) and any helper files. You read it over and fix anything before it goes live — it can flag a description too vague to trigger reliably.

3. **Save it into your project** Put the folder under .claude/skills/<name>/ in your project (the folder beside your project's CLAUDE.md spec). The whole group gets it once it is committed, and the agent loads it automatically whenever a task matches its description.

4. **Or write the SKILL.md by hand** It is just a folder: make one, drop in a SKILL.md with the name-and-description header and your instructions, add a script if a rule must run in code. Good first skills for this course: a writing-standards skill that holds your report tone and format, or a column-check skill like Kinquiry's codebook-guard.

> **Describe the skill, let skill-creator scaffold it, review it, then save it under .claude/skills/.**

One skill, three jobs: it can block, advise, or render

HARD GATE

codebook-guard

A hard gate that blocks the run. Its validate.py checks columns against the 145-column survey list, stops if one is invented, and refuses to average the Q3.1_* agreement scale with Q5.1_* performance.

ADVISORY

interpretation-guardrails

Advisory: it flags problems but lets you proceed. Its check_claim.py linter reads a draft and flags causal wording, a missing effect size, or a missing caveat. It helped produce beta 0.16, N 2,180.

DETERMINISTIC

presentation-render

Deterministic: same input, same output every time. It stages a written claim in a fixed order (finding, sample, estimates, robustness, contribution, key assumption) and suppresses groups under 10 firms.

Three of Kinquiry's six jobs took the skill form, each a different posture; the skill is not the only option.

FORM 2 OF 4

Subagents

A separate worker you hand one job, used to isolate a single decision worth isolating.

A subagent is a scoped helper in its own window

- **What a subagent is** A single-job helper: hand it one task, get one answer. It runs in its own context (the text a model reads at once), sees none of your main session, returns only its result.
- **Why the isolation matters** A side job reading many files would crowd your main session. As a subagent it runs separately: Kinquiry's hypothesis-parser reads the survey file there, not the build thread.
- **It can run a cheaper model** When a job is narrow and mechanical, point the subagent at a smaller, cheaper model to cut cost. Checking column names against a list needs no top model (Anthropic, 2025b).
- **Build one per decision worth isolating** Isolate a subagent only where a real judgment is made, not at a routine step. Kinquiry has two: turn a rough hypothesis into a plan or a refusal, and write the claim.

> **Count the decisions worth isolating, not the steps; extra workers add cost and risk without adding judgment.**

Kinquiry's two subagents: hand, return, consult

WORKER 1 · PARSE

hypothesis-parser

HAND IT the researcher's rough question. IT RETURNS a validated plan (outcome, predictor, controls, estimator) or an honest refusal with a reframe. Nothing runs until that plan is approved.

IT CONSULTS

codebook-guard

The worker carries the decision; the skill carries the rule. Before naming any column the parser runs codebook-guard, so it can never propose a field STEP does not contain among its 145 real columns.

WORKER 2 · WRITE

interpreter

HAND IT the finished numbers (estimate, sample size, robustness). IT RETURNS the one written finding: the effect size in plain words plus the required caveats. The only component allowed to write a claim.

IT CONSULTS

interpretation-guardrails

The interpreter runs its draft through this advisory skill, which flags causal wording ('causes' not 'associated with'), a missing effect size, or a dropped caveat. It advises; the writer rewrites.

No third subagent renders the result: a fixed render skill (presentation-render) does it, since it isolates no decision.

THE EVIDENCE, WITH ITS CAVEATS

Multi-agent setups score higher — by spending far more

+90.2%

A lead agent delegating to subagents beat one agent (Claude Opus 4) by this on the test.

Anthropic, 2025d

~15x

Tokens (billed units of text) the multi-agent run used vs a chat. Spend explains 80% of it.

Anthropic, 2025d

The lift fits breadth-first work, not dependent steps; Kinquiry's chain is dependent, so two subagents suffice.

FORM 3 OF 4

Connectors

A wiring that lets the agent read and act in your real tools and data, instead of you pasting them in.

FORM 3 OF 4 · CONNECTORS

A connector lets the agent reach an outside tool or data source directly

- **What a connector is** A connector is a standing link you set up once so the agent can read from and act in an outside system — your email, a file store, a code repository — directly, instead of you copying a piece of it into the chat by hand.
- **MCP is the standard underneath** Connectors run on the Model Context Protocol (MCP), an open standard Anthropic released in late 2024 for connecting an agent to outside tools and data. Each system exposes itself through a small program called a server that the agent talks to.
- **Think of it as a universal adapter** MCP's own documentation likens it to a USB-C port for AI: one shared standard, so the agent plugs into many systems the same way. Before MCP, every tool needed its own custom wiring; now one protocol connects them all.
- **Configure it; you rarely build one** Thousands of connectors already exist, so for common tools you click connect and sign in rather than write code. And one you set up in Claude carries over when you log back in, so you never wire the same source twice.

> **A connector is a setup step, not a coding job: pick the ready-made one, authorize it, and the agent works the real system directly.**

CONNECTORS YOU CAN ACTUALLY USE

Four common connectors — and the one I used to email a collaborator

GMAIL

Read, draft, and send mail

The agent works your inbox directly — search and read messages, draft a reply, and send it.

GITHUB

Read and act on a codebase

The agent reads your repositories, issues, and pull requests, and can open or merge changes.

GOOGLE DRIVE

Find and read your files

The agent searches your Drive and reads your real documents, so answers come from your files.

SLACK / CALENDAR

Pull team context, manage time

Slack lets the agent read and post in channels; Calendar lets it check availability and book time.

REAL EXAMPLE

Emailing a Kinquiry update through Gmail

With the Gmail connector on, Vern had Claude write and send Jen Kotadia a formatted HTML “Kinquiry is live” update.

THE TEACHING POINT

The agent acts; the human reviews the send

Claude acted inside Gmail, not just drafted — and the reviewed step was the send, which Vern approved.

KINQUIRY

The connector Kinquiry chose not to build

A connector lets the agent reach a live outside system, such as a hosted database or statistics service, while it works. Kinquiry needs none, so it builds none, and that refusal is the instructive part. Its data is one local file (the STEP family-business survey of 2,683 firms). The statistics run as Python code that reads that file directly, and the output is a research presentation written to another local file. Local data in, local file out: at the moment the agent does its analysis, there is no external system to reach, so the connector form stays empty on purpose.

This holds even though Kinquiry is fully deployed and live. Its web interface runs on Vercel and its statistics service on Render, both publishing automatically from the main branch, all version-controlled on GitHub at kinquiry.vernglaser.com. But that is infrastructure for shipping and storing the finished app. A connector is something the agent calls while it is doing the work. None of GitHub, Vercel, or Render is called mid-analysis, so the form is genuinely empty.

> **A connector earns its place only when an app's value rests on reaching a live outside system. When nothing does, leave the form empty.**

FORM 4 OF 4

Dynamic workflows

Letting Claude orchestrate its own steps for a big task — recognize it; for this build you mostly leave it alone.

FORM 4 · DYNAMIC WORKFLOWS

Three ways to control one part of your app

Default harness

A harness is the scaffolding around the model — how a task is planned, split into steps, checked, and run. Claude Code's default harness is built for coding and carries most of a build by itself. Kinquiry's research agent ran on it alone; no extra scaffolding was built.

Dynamic workflow

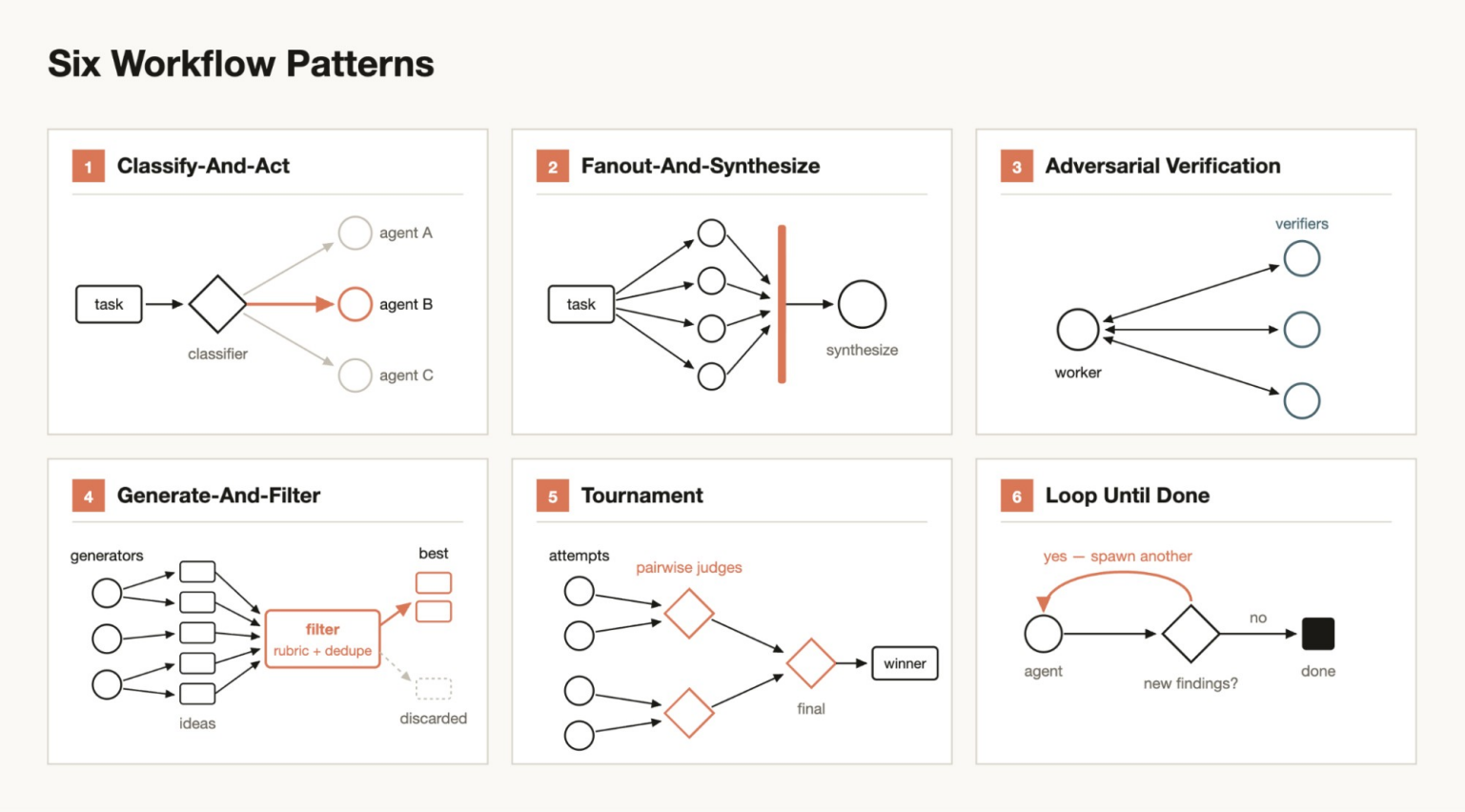
Here Claude writes its own harness on the fly, custom-built for the task, chaining steps a single straight-through pass cannot do. It is capable but costly in tokens (the units of model usage a build pays for). Kinquiry's separate dashboard was built this way: nine workers, about 640,000 tokens.

Fixed pipeline

You hard-code the steps in one set order, so the same inputs always give the same outputs. Kinquiry's analysis always runs clean, then test, then effect size, then reliability, then robustness — and its validate.py exits with an error rather than average the two 1-5 scales, a rule enforced in code.

Start with the simplest of the three. Add a dynamic workflow only when one part genuinely needs it.

Six ready patterns a dynamic workflow can assemble



A pattern is a reusable way to arrange several model steps one straight pass cannot do. One Kinquiry example: adversarial verification spawns a skeptic per finding to confirm it never claims causation and always reports an effect size.

When a workflow is worth it, and how you get one

- **What a dynamic workflow is** Claude writes its own multi-step plan for a job too big for one pass, then a runtime runs it using many subagents (scoped workers). Capable, but the most costly form.
- **When it is worth the cost** Use one only for complex, parallel, or high-value work one pass cannot do; it spends far more tokens (the billed units of model usage). For a one-week build, just awareness.
- **How you get one: you ask** You ask Claude to set up or run a workflow, and Claude writes the orchestration. You do not write it: you describe the job and approve the plan it proposes before it runs.
- **What the keyword ultracode does** Typing ultracode opts that request into workflow orchestration; it signals you want one. It does NOT build the workflow; Claude still writes it. Plain words work the same.

> **The payoff is in Part III: Evaluate's skeptic-per-finding checks are the parallel, adversarial work a workflow is for.**

KINQUIRY

Same project, two answers: a dashboard built by nine workers

Kinquiry made the workflow choice twice and answered it differently each time on the same STEP survey data. Its research agent used no workflow at all — the default harness already supplied what it needed, and the analysis was a single ordered run one straight pass could handle. But Kinquiry's separate dashboard — eight cross-filtered sections (panels where filtering one updates the others) plus a landing page — was built by a nine-subagent workflow. A subagent is a scoped worker with its own separate context that does one isolated job and reports its result back.

Before any worker ran, the instructor hand-built a fixed contract: the exact shape of each file and how the pieces fit together, with empty stubs so the whole app compiled first. With that contract in place, the nine workers ran at the same time, each writing one file without colliding with the others. The work was genuinely parallel and near-independent — which is exactly when a workflow earns its cost.

The honest caveat travels with it: the nine workers spent about 640,000 tokens, a large and deliberate spend. That cost is why a workflow is something you choose on purpose for the right job, never something that happens automatically.

> **A workflow earns its tokens only on genuinely parallel, near-independent work.**

WORKING METHOD

Manage the build session

The live session has a limited memory; keep it lean so the agent stays accurate.

Your live session is a budget you have to manage

- **What the session is** The session is the continuous Claude Code conversation you build in. Everything in it sits in the model's working memory and is re-read every turn. That memory is limited, so as it fills, the model attends less to any one instruction and starts forgetting earlier ones.
- **Bring the least text that still works** Carry the smallest high-signal slice, not the whole file. Pasting a 40-page document so the one line that matters drowns among 39 that don't is the classic waste, paid for again each turn. Kinquiry's STEP codebook was extracted to a tight text file once, never re-pasted.
- **Keep the spec lean too** The CLAUDE.md is the durable spec the agent re-reads every session. A bloated spec gets ignored the same way a bloated conversation does, and you pay for it every run. Keep only what must persist; drop the rest.
- **Reset on purpose; push work outward** /clear wipes the conversation to start fresh; /compact summarizes it to keep the gist while shedding bulk. A subagent is a separate worker with its own memory: Kinquiry's hypothesis-parser reads the files itself, so the files it opens never enter your conversation.

> **After a couple of failed corrections, a clean session with a better prompt beats a long one carrying its confusion.**

KINQUIRY

Three moves that keep Kinquiry's build session lean

Kinquiry is the instructor's research workbench that runs statistical analyses on the STEP family-business survey of 2,683 firms. Move one: convert a heavy, reused source to text once. The survey codebook (the reference listing all 145 columns and what each measures) was extracted a single time into a tight text file the agent is pointed at, instead of re-pasting the whole codebook every turn. The codebook-guard skill then wraps that file in a check: its script, validate.py, confirms every column an analysis names exists in the 145-column manifest, and refuses any composite that averages the survey's two different 1-5 scales.

Why that refusal matters: the legacy items (columns like Q3.1_6) run from 'strongly disagree' to 'strongly agree,' while the performance items (Q5.1_1) run from 'much worse' to 'much better.' A 5 on one is not a 5 on the other, so averaging them produces a meaningless number. The rule lives in the skill, not in the builder's head.

Move two: /clear (the command that wipes the conversation's memory) is run between unrelated test runs, so a new question does not start cluttered with the last one's exploration. Move three: the file-reading and column-checking that would otherwise flood the conversation run inside the hypothesis-parser subagent — a separate worker that turns a rough question into one validated plan (such as the plan behind the gold result, beta approximately 0.16, 95% CI [0.11, 0.20], N approximately 2,180) and returns only that plan.

> **The bulk stays out of your conversation: it lives in a skill, in a subagent, or cleared away.**

ACTIVITY

90 min · groups

Map your app's capabilities, then build at least one

Bring the two halves of Build together: first map what your app must do and how, then build at least one of those capabilities for real.

- **Part A · Walk the skeleton, list the verbs** Go through your walking skeleton (the HTML mock of every screen with its content faked) one screen at a time. At each faked spot, write what the agent must be able to do as a plain verb. Name no tools yet, so the list grows by what the app needs.
- **Part A · Choose a form for each verb** For each verb pick the simplest form that does the job: native action, skill, subagent, or connector. Write the verb beside its chosen form. That paired list is your capability-to-form map — the first half of today's output.
- **Part B · Build one capability in its form** Pick one verb from your map and build it now. Have Claude package a skill or scope a subagent; for a connector, configure an existing one rather than hand-build it (thousands already exist). This is the half most groups skip — do not stop at the map.
- **Part B · Wire it in and check behavior** Install it in the project (a skill goes in .claude/skills/<name>/, not pasted in chat) so the agent finds and runs it on its own. Then feed it a known input and confirm the output is what you specified — the way Kinquiry's codebook-guard runs validate.py on an invented column and exits with an error.
- **Verify, don't just trust** A tool reporting success is not the same as the right answer. Check what the capability actually produces against a case whose correct result you already know, and be able to explain in plain words what it does. That behavior check, not a green status, is the proof it works.

OUTPUT **A capability-to-form map (each verb beside its chosen form) plus at least one working capability installed in the project and behavior-checked on a known input.**

SHIP

CHAPTER 7

Building the App

Have Claude write the app from your spec, test it by using it, and ship it through GitHub.

COMMON FAILURE

Every number was correct, and the app still shipped broken

This is a real episode from Kinquiry, the instructor's research app: a user types a research question and the app runs a statistics regression behind the scenes, showing the result as charts. He ran Kinquiry's reference 'gold' question (do stronger family social ties go with better firm performance?), and it came back right: a coefficient of about 0.16 across roughly 2,180 firms. He pushed it live.

Then the deployed app betrayed him. He opened the live page on his phone, and the result never came into view: the page would not scroll past the input box, so the answer was there but unreachable. On a laptop, the coefficient plot (a horizontal bar whose length shows the 0.16 result) ran off the right edge of its frame, because the chart's number axis was fixed to a range too narrow for this real finding.

Neither defect touched a single computed value. Every number the app produced was correct. What shipped broken was the app around the numbers, where a value-by-value check (the discipline of the previous chapter) never looks.

> **The build's last check is not any one output. It is the running app, used the way a real person uses it.**

Ask Claude to build the app; judge it by what it does

- **What vibe coding is** Vibe coding, a term Andrej Karpathy coined in early 2025, means describing what you want in plain words, letting the model write the code, and steering by what it returns.
- **Build from your spec, in plain words** You ask Claude to build the app from your spec (the CLAUDE.md): the page a user sees, what asking does, what the result looks like. You say what is wrong in plain words.
- **Verify, then trust: judge behavior** You never read the diffs — the line-by-line code the model writes. You check what it does: ask a question whose answer you know, confirm the number; open the page, scroll it.
- **A deployed app clears Willison's bar** Karpathy bounded vibe coding to throwaway projects. A deployed app others rely on is not: Willison's line is you must test it, stand behind it, and explain it to someone.

> **You are a manager directing a build, not an engineer auditing one. A deployed app is not a throwaway project.**

The one decision you own: what to build the app with

- **This call is yours.** Claude writes the code, but what the app is built with (the stack) and where each part runs (the infrastructure) is a judgment about cost and complexity — choose it on purpose.
- **Front end vs. back end.** The front end runs in the browser, which speaks only JavaScript; a back end runs on a server. A browser cannot run Python's statistics — pandas, statsmodels, a real regression.
- **The one question that decides it.** Does the app need heavy computation a browser can't do? Yes → a separate Python service. No → one program is enough. Everything downstream follows from that answer.
- **Most dashboards can skip it.** Do the heavy work once, offline; ship a prepared file; filter and chart it in the browser. Kinquiry's explorer does exactly this — 2,683 firms, no server behind it.

> **Heavy live computation a browser can't do? That one answer sets your whole infrastructure.**

Three ways to build the infrastructure

ALL-IN-ONE PYTHON

Streamlit

One Python program does the page and the computation — no JavaScript, no separate front end. Fast and pure-Python; you accept its look and limited control. (Dash and Gradio are cousins.)

ONE WEB APP

Next.js on Vercel

A polished, custom page (React) with a thin server layer for the Claude API call. One project, one deploy — but no heavy Python. The realistic target for most projects.

TWO TIERS · THE STRETCH

Next.js + Python service

The web app plus a separate FastAPI + statsmodels service (on Render) for real statistics. Production-grade and genuinely inferential — but a second service, shared secrets, and cold starts.

Fast in pure Python → Streamlit. Polished web app → a single Next.js app on Vercel. Real live statistics → add the Python service.

The build stack at a glance

Streamlit	A Python library that turns a Python script into a web app. Pick it for a data dashboard you build entirely in Python.
React / Next.js	React builds a custom interface from reusable components; Next.js is the production framework around it. Pick it for a polished web app.
FastAPI (Python)	Exposes Python functions — a statsmodels regression — as a service the front end can call: the back end for real computation. (Flask is the minimal alternative.)
Hosting	Vercel for the web front end; Render or Railway for a Python service; Streamlit Community Cloud for Streamlit. Free tiers sleep when idle, so the first request is slow.
Charts	Claude draws them with a library (Recharts in React, Chart.js in plain pages, Plotly in Python) or by hand — an implementation detail, not your call.

Pick the setup the app actually needs, not the most elaborate one.

KINQUIRY

Kinquiry's stack — the stretch, not the floor

Kinquiry took the two-tier path, and the course treats it as a deliberate stretch, not the target. Its page is a Next.js / React app on Vercel; its reasoning routes call the Claude API server-side to sharpen a question and interpret a result; and its statistics run in a separate Python service — FastAPI with statsmodels — on Render.

The browser only ever talks to Vercel, which forwards an analysis request to the Python service server-side, so the secret keys never reach the user's machine. That buys real, reference-grade inferential statistics; it costs a second service, secrets shared across tiers, and a cold-start wait on the free tier.

The realistic one-week target sits one rung simpler: a client-side dashboard plus a thin Claude reasoning layer, shipped as a single Vercel app with no separate statistics service. Know what the second tier buys, and what it costs — you do not have to build it to ship.

> **You choose the stack and judge the app by behavior — you never write its code.**

Four ways to verify a deployed app by using it

1 Pin a regression oracle

An oracle is one input whose correct answer you have established as truth. Hit the live app with it and confirm it returns. Kinquiry's oracle: about 0.16 over ~2,180 firms.

2 Exercise it like a user

The oracle confirms one path. Confirm the rest by using the app as a person will: varied questions, every screen, phone and laptop. This surfaced Kinquiry's unscrollable page.

3 Judge plausibility

When a number is computed right but cannot be true, your judgment catches it; a passing test only certifies the code ran. Kinquiry showed a firm of 300,290 employees, impossible.

4 Get a cold read

You cannot un-know what you built, so you are the worst person to spot where it confuses a newcomer. Hand it to a fresh pair of eyes; a cold read finds dead ends and dud buttons.

Managers judge behavior, not code. The worst defects live in the product around the numbers.

KINQUIRY

The oracle reproduced, and a number that couldn't be true

Kinquiry's oracle is its gold question: does a firm's binding social ties (a STEP-survey measure of family cohesion) relate to how it performs against rivals? The certified answer is a small positive association of about 0.16, 95% CI [0.11, 0.20], over roughly 2,180 firms, an association, not proof of cause. The instructor asked exactly that of the deployed app, confirmed 0.16 came back, then clicked through every screen.

In Kinquiry's data explorer (the raw-data front door, where a person scans actual survey rows) he saw a firm listed at 300,290 employees in a $20M-$50M sales band. No firm that small employs three hundred thousand people, so he flagged it and traced it to a typo: the true count was 290. The charts winsorize the outlier (cap an extreme value so it cannot skew a result); the explorer shows the raw value on purpose.

That split is deliberate, and it is why the bad row was caught at all. The system did what it was told: capped the outlier in the charts, kept it raw in the explorer.

> **Finding the bad row proved the build worked as specified, instead of exposing a defect.**

GitHub: where the app lives, ships, and is reviewed

The app Claude built is code, and that code lives in GitHub: a system that tracks every change to the app's files and lets a team work on them without overwriting each other.

Repository	The shared project folder for the app, except it keeps the full history of every change. Kinquiry's code lives in one repository, and pushing to it is also how the app deploys.
Branch	Your own private parallel copy of the project. You edit it freely without touching the official version everyone else relies on.
Commit	A labeled snapshot you save as you go. As Stulberg and Koh put it, a commit does not mean done; it means a meaningful checkpoint. On Kinquiry, one was saved as each capability came together.
Push / pull	Push sends your changes up to the shared repository so teammates can see them. Pull brings down the latest changes your teammates have already pushed.
Pull request	You raise your hand that a change is ready, and a person reviews it before it joins the official version that real users reach. It is the review step every change must pass.
Main	The official, trusted version everyone relies on. A change merged into Kinquiry's main auto-deploys to the live site at kinquiry.vernglaser.com, so main is what the public sees.

The daily loop: pull main, branch, edit, commit, push, open a pull request, merge. Because a push to main serves the app, the repository is also the deploy button.

KINQUIRY

The pull request is the one review every change must pass

This is the real governance arrangement from the instructor's Kinquiry build. Kinquiry's repository is private because it carries the proprietary STEP survey data (the SPGC-KPMG Global Family Business Survey, 2,683 firms across 83 countries). On a free private repository, GitHub will not let you enforce branch protection: the platform rule that forces every change to main to pass a pull-request review before it can merge. There is no technically binding way to require that review.

Why that matters: a push to main automatically rebuilds and serves the live site (the Next.js app on Vercel and the statistics service on Render at kinquiry.vernglaser.com), so a collaborator with write access could push straight to main and change what every visitor sees, with no one looking first. So the instructor set the collaborator's access to read only and had her contribute by fork (her own separate copy of the repository, with no write access to the original) plus a pull request he reviews and merges.

That pull request is the Governance membrane. Governance is the boundary every input and output crosses under human review, so the pull request is the single point where every change passes a human before reaching users. Claude does the work on the branch; a human owns what crosses into main. The review the free tier could not enforce, he enforced by workflow.

> **Delegate Work, Not Accountability: agents do the work, but every change reaching real users passes one human review first.**

ACTIVITY

75 min · groups

Build, test, and ship the app

Direct the build from the CLAUDE.md your group wrote on Day 1: the living spec stating your app's objective, scope, and success criteria.

- **Build it from your spec** Ask Claude to build the app from your CLAUDE.md. When something is wrong, say in plain language what the app does wrong. You judge what the app does, not the code Claude wrote to do it.
- **Confirm your known answer** Hit the live app with the one question whose correct answer you already certified, the way the instructor hit Kinquiry's live route and got back $\beta \approx 0.16$, $N \approx 2{,}180$. Confirm the same answer comes back.
- **Use it like a stranger** Push varied and awkward questions through every screen, on a phone and a laptop. Then have someone who has never seen the app try it; you know what it should do, so you are worst at spotting where it confuses a newcomer.
- **Ship it through review** When you push your changes, the host automatically rebuilds the app and serves the new version at its live URL, the way Kinquiry deploys to Vercel and Render. Mirror Kinquiry: every change to main passes one human review first.
- **Start the deployment log** Open a structured record of salient episodes, overrides, failures, and reactions from people outside the group, and carry the app into the one-week Use phase. The log captures what the week reveals that the build could not.

OUTPUT **A deployed, version-controlled app in real Use, with the deployment log started.**

The build is done; the week of real use reveals the rest

You now have the capabilities built and assembled into a deployed app — one people outside your group can open in a browser — and you have checked that it behaves the way your spec said it should.

You are not building an evaluation today. Measuring the app against your spec is Day 3's job, after it has been used. Today ends by putting it to work.

Carry the app into one week of real use, and keep a deployment log: the moments it surprised you, the times you overrode it, anything that broke, and how people outside your group reacted.

> **Ship a working app into real use, write down what happens, and bring that back for Day 3.**